

Lumielle Academic Assistant

LIGHTWEIGHT · LOCAL · ONE-CLICK

Academic Paper Writing Assistant

From Research Topic to Final Draft — One-Stop Experience

1. Introduction & Why We Built It

Web-based AI chat is powerful, but writing a long paper comes with a few unavoidable pain points:

- **Long-conversation memory loss:** the longer the chat, the more the AI "forgets" earlier context — or even invents it.
- **Hard to revise repeatedly:** if a result isn't satisfactory, you have to regenerate it from scratch instead of editing the draft directly.
- **Context bloat:** every revision sent back into the chat makes the context longer, messier and more expensive.

Lumielle Academic Assistant turns the whole writing pipeline — **Topic Setup** → **Literature Management** → **Logic Outline** → **Chapter Writing** → **AIGC Detection & De-smelling** → **Final Word Export** — into a local tool. The AI focuses on writing one chapter at a time, while the tool manages the connections between chapters automatically, giving the model exactly as much context as it needs — never unlimited.

Design Highlights

- **Lightweight:** runs locally, no server needed, one-click deployment.
- **Local & private:** all data stays on your machine, never uploaded.
- **Smart context management:** a "context sandwich" that always gives the model the right amount of context.

2. Installation & Deployment

1 Get the package

MacOS / Windows versions. Unzip, then double-click to launch — dependencies install automatically.

2 Apply for an API Key

Apply at the DeepSeek open platform (or any OpenAI-compatible service).

3 Start using it

Double-click **Launch.command**, wait for the first-run setup, then enter your API Key in the app.

Package Layout

- **Launch.command** — one-click launcher (double-click)
- **app.py** — main program
- **.uv/** — uv runtime (auto-prepared)
- **.python/** — bundled Python 3.12 runtime
- **requirements.txt** — dependency list
- **data/** — data directory (auto-created; delete to reset)
- **models/** — optional local AIGC model (LLM engine used by default)

💡 **First launch:** the tool creates a virtual environment and installs dependencies (about 1-3 min, needs internet). After that, it starts in seconds.

3. Who It's For / Not For

✓ Great for

- Academic writing newcomers who need to quickly build a paper framework and generate a first draft.
- Undergraduate thesis and course paper writers.
- People tired of copy-pasting between AI chat windows.
- Those who want a lightweight, local and private writing environment.

✗ Not ideal for

- Deep scientific research that requires complex experiments, extensive literature databases and strict data validation.
- Those expecting fully automated, submission-ready papers without any manual revision.

4. Quick Start (5 Minutes)

- 1 Set the topic** — fill in the research topic and target word count in the sidebar. This is the global foundation; do it first.
- 2 Configure LLM** — enter your DeepSeek API Key → fetch models → pick a model.
- 3 Build the research base** — fill in research background and topic framework.
- 4 Generate** — import literature → generate outline → write chapters → export Word.

5. Recommended First-Time Path

Follow this order once to learn each module, then combine them freely to suit your workflow.

1 Sidebar global settings — set topic & length (do this first!)

Fill in the research topic and target total word count in the left sidebar.

⚠ This is the foundation of every module — the research base, logic outline and chapter writing all build on these two settings.

2 LLM Configuration — connect the "brain" first

Enter your DeepSeek API Key, fetch the model list, pick a model.

⚠ Enter the key before fetching models, otherwise it will error.

3 Research Base — build the framework

Fill in research background and expected outputs; split the research into modules. This is global context — fill it carefully, everything builds on it.

4 Literature — import your ammunition

Drag in whole folders or single PDFs; the tool parses, rates and classifies them.

5 Logic Outline → Chapter Writing → De-smell → Export

Generate the outline, write chapters, run AIGC de-smelling, and export the final Word document.

6. Module Guide

6.0 Sidebar Global Settings (do first)

The first thing after launching: fill in the **Research Topic** and **Target Total Words** in the left sidebar. This is the global foundation — the research base, logic outline and chapter writing all generate content based on these two settings.

6.1 LLM Configuration

- Enter your DeepSeek API Key (Base URL is pre-filled with the official endpoint).
- Fetch the model list, then pick a model — the flagship reasoning model is recommended (strong inference, good de-smelling results).
- Save the configuration and you're ready.

6.2 Research Base

The starting point of your research. Set up the overall framework here:

- **Research background:** topic, background materials, expected outputs.
- **Module management:** split the research into modules and fill in content requirements one by one.

All settings here are synced to the logic outline and chapter writing as global context.

6. Module Guide (continued)

6.3 Literature Processing

- **Batch upload:** drag in whole folders or single PDFs.
- **Auto parsing:** extracts title, abstract, authors and more.
- **Rating & classification:** each paper is rated and categorized by the LLM; chapter writing only cites papers that passed the rating.

6.4 Logic Outline (core module)

The logical skeleton of your paper is built here, with multi-level outline support:

- **Outline generation & editing:** auto-generate a multi-level outline; every node can independently edit title, description and word count; add/delete nodes freely.
- **Node editing:** parent nodes act as chapter containers; leaf nodes are the actual content entities.
- **Chart instructions:** each leaf node can be configured with chart-drawing instructions; the LLM auto-generates charts and mounts them to the chapter.
- **Personal experiment marking:** chapters marked as personal experiments are treated as original research / first-hand data when generating text.
- **Word count management:** per-node word allocation at a glance, with whole-tree statistics to keep length under control.

6. Module Guide (continued)

6.5 Chapter Writing

Single-chapter writing & editing:

- **Chapter navigation:** written chapters show a  mark; parent nodes show subtree progress at a glance.
- **Global context:** before generating, the topic / outline / upstream memory / downstream boundary / bound references are all displayed.
- **AI generation:** auto-corrects word count; compresses memory after generation for later chapters to stay consistent.
- **Charts & images:** charts are auto-generated and mounted; images can be uploaded with explanations.

Batch workbench:

- Tree-style collapsible navigation; parent nodes are containers, only leaf nodes can be queued.
- Two independent queues: chapter generation / chart drawing, with one-click select-all.
- Batch-generate chapters with per-chapter word-count correction and a live progress bar.

6.6 AIGC Detection & De-smelling

- **Detect:** pick a chapter and click Detect to see the AI probability percentage. Two engines available (LLM / local model).
- **De-smell:** select chapters with high AI probability and click De-smell; the tool polishes them repeatedly until the AI flavor drops, then shows the result. A manual read-through after de-smelling is recommended.

6.7 Prompt Configuration

The global "flavor adjustment zone" controlling the paper's style and de-smelling strength. Defaults are tuned for academic scenarios — newcomers can leave them as-is.

7. FAQ

Q: Where is my API Key stored?

A: Only in the local configuration file; it is never uploaded to a server.

Q: Will my data be lost?

A: Your topic, literature, and drafts are stored in the local data/ directory and persist after normal exit.

Q: Are the detection results reliable?

A: Results are lightweight assistance for reference; cross-check with professional platforms for important decisions.

Q: Do I need an internet connection?

A: Only for the first dependency installation and LLM API calls; the application runs locally.

Q: Can I reset all data?

A: Delete the local data/ folder and relaunch.

8. Data Safety

Your paper content stays in the local data/ directory and is not uploaded to any server. The tool runs on your machine.

9. Open-Source License

Lumielle Academic Assistant is an open-source project released under the MIT License.

GitHub: <https://github.com/Lumielle-BlueOMOcean/Lumielle-Academic-Assistant>

Community & feedback: QQ Group 1029688024

Developer: Lumielle



Finally, the developer insists on showing off this beautiful poster.

— No wait, it's *me* who insists. After all, who could resist me looking this good? ✧

Appendix: Version History

v1.0.1 — 2026-09-27

Added

- A single SemVer source in version.py now supplies the version displayed in both applications.
- Focused regression tests and Python 3.12 GitHub Actions CI now guard core behavior.
- Drafts retain reference snapshots and mappings so citation exports stay consistent over time.
- A safe chart pipeline validates structured chart specifications and renders them with trusted local code.
- Research Foundation handles complete documents in ordered chunks; Manuscript Reverse Engineering processes whole manuscripts with ordered extraction and bounded consolidation.
- Both application sidebars identify the MIT open-source license, GitHub repository, and QQ community.

Changed

- Chapter generation now receives the Context Sandwich global outline, upstream context, and downstream written content or boundaries.
- The user-saved writing-style prompt is interpolated into each chapter-generation prompt.
- English counts use word tokens for generation correction, UI and batch results, structure review, and memory status.
- English defaults initialize inside prompts.json without replacing an existing user prompt file.
- Word export globally orders and deduplicates references, remaps chapter-local citations, and uses saved snapshots with a legacy-binding fallback.
- Long Research Foundation documents use ordered chunks instead of silently truncating the beginning; long manuscripts use ordered extraction and bounded consolidation before replacing the outline.
- Multi-model consensus uses successful answers only; a failed outline operation leaves the previous outline intact.
- Literature remains imported when AI rating is unavailable and is marked Unrated; failed deep-review blocks are excluded, and an all-failed review preserves the prior report.
- MIT licensing information is consistent across LICENSE, README, manuals, and both application interfaces.
- Batch chapter generation reports individual failures and continues with the remaining chapters.

Fixed

- Fixed the Context Sandwich display/runtime mismatch so its context constrains the actual chapter-generation request.
- Fixed the literal {style_prompt} placeholder being sent instead of the saved custom style.
- Fixed English word counts being measured as non-whitespace characters.
- Fixed English default prompts being initialized at the wrong configuration level.
- Fixed macOS first-run environment creation by using bundled uv to create a pip-enabled environment because standalone Python omits venv and pip.
- Fixed cross-chapter citation numbers by using one global ordered reference registry during export.
- Empty responses and known LLM failures are rejected before they can be saved as generated content.
- A failed chapter-memory distillation no longer overwrites the saved draft or previous summary.
- Fixed chart execution safety by replacing arbitrary model-generated Python execution with validated chart specifications and trusted rendering.
- Illustrative charts require explicit permission; negated wording does not authorize fabricated values.
- Fixed silent 5,000-6,000-character prefix truncation in Research Foundation parsing.
- Fixed silent 4,000-character truncation in Manuscript Reverse Engineering.
- Chinese and English document extraction failures now receive consistent handling.
- Configuration, API, and empty-output failures cross one checked LLMOutputError boundary and stay out of drafts, outlines, and reports.
- Failed literature rating no longer appears as a neutral three-star or Other result; it is reported as unavailable and Unrated.
- AIGC detection failure now reports unavailable instead of a misleading 0% AI.
- Failed logic-review blocks are excluded from normal reports, and an all-failed run preserves the previous report.
- Existing unreadable/corrupted user JSON files are now preserved and further writes to those files are blocked instead of silently replacing them with defaults.
- Failed model answers no longer affect multi-model consensus, and outline failures no longer discard the prior outline.

Removed

- Removed arbitrary execution of LLM-generated Python for chart rendering.
- Removed silent fixed-prefix truncation from Research Foundation and Manuscript Reverse Engineering.
- Removed README and UI non-commercial restrictions that conflicted with the MIT License.
- No core user-facing workflow was removed in v1.0.1.

v1.0.0 — 2026-08-26

Initial public release with Chinese and English desktop packages for macOS and Windows.

Open-Source Project

Lumielle Academic Assistant is an open-source project released under the MIT License.

GitHub: <https://github.com/Lumielle-BlueOMOcean/Lumielle-Academic-Assistant->

Community & feedback: QQ Group 1029688024